

蓝桥杯30天算法冲刺集训



Day-28 数论II

扩展欧几里德算法

裴蜀定理

裴蜀定理(贝祖定理、Bézout定理): 对于任意的不全为零的整数 a, b , 不定方程 $ax + by = c$ 对 (x, y) 有整数解的充要条件为 $\gcd(a, b) | c$, 特别的, 一定存在整数 x, y 满足 $ax + by = \gcd(a, b)$ 。

我们先来证明: 一定存在整数 x, y 满足 $ax + by = \gcd(a, b)$ 。

证明一:

先证 **充分性**: 如果 $\gcd(a, b) | c$, 那么 $ax + by = c$ 一定有整数解。

我们只需要证明一定有一组 x_0 和 y_0 使得 $ax_0 + by_0 = \gcd(a, b)$ 就可以了, 这样对于任意的 $k \in \mathbb{Z}$, 等式 $akx_0 + bky_0 = k \times \gcd(a, b)$ 一定成立, 这样充分性就证明完成了, 下面我们来证明存在这样的 (x_0, y_0) :

设集合 $A = \{c | ax + by = c, x, y \in \mathbb{Z}\}$, 即 A 是 $ax + by$ 所以能取到的值的集合。

先证: 如果集合 A 不空, 那么其中最小的正元素即为 $\gcd(a, b)$ 。

我们假设集合 A 中的最小的正整数是 s , 其对应的一组正整数解为 (x_0, y_0) , 则有 $ax_0 + by_0 = s$ 。

顺着思路来, 要证 $s = \gcd(a, b)$, 只需证

$$\begin{aligned} \gcd(a, b) &| s \\ s &| \gcd(a, b) \end{aligned}$$

因为 $\gcd(a, b) | ax_0$, 且 $\gcd(a, b) | by_0$, 所以, 根据整除的性质, 显然有 $\gcd(a, b) | s$ 。

如此, $\gcd(a, b) | s$ 的性质我们证毕。

接下来证明 $s | \gcd(a, b)$ 。

设 $a = qs + p (0 \leq p < s)$, 则 $p = a \bmod s$, 于是有:

$$p = a - qs = a - q(ax_0 + by_0) = a(1 - qx_0) + b(-qy_0)$$

于是我们发现 p 可以写成 $ax + by$ 的形式, 也就是说 p 也是集合 A 中的元素, 又因为 s 是集合 A 中的最小正整数, $p \in [0, s)$, 所以 p 只能为 0。

如此, 综上, 充分性证毕。

再证 **必要性**: 如果 $ax + by = c$ 有整数解, 那么必有 $\gcd(a, b) | c$ 。

$$\begin{aligned} c \% \gcd(a, b) &= (ax + by) \% \gcd(a, b) \\ &= ax \% \gcd(a, b) + by \% \gcd(a, b) \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

所以, $\gcd(a, b) | c$ 。(必要性的成立也可用反证法, 如果 c 不能整除 $\gcd(a, b)$, 那么等式 $ax + by = c$ 必然无法成立)

证明二: 归纳法

来看欧几里得算法求解过程的这个函数:

```
int gcd(int a, int b) {  
    return b == 0 ? a : gcd(b, a % b);  
}
```

可以看出, 这是一个递归求解的函数。在函数递归到最后的时候, 存在 $b = 0$, 不管 a 是什么, 这时显然有一对整数 $x = 1, y = 0$ 来使得:

$$a \times 1 + 0 \times 0 = \gcd(a, 0)$$

(备注: 0 和任何数的最大公约数都等于原数, 可以从最大公约数和约数的定义得知)

(归纳法) 那么, 我们通过这个递归的实现过程来进行回溯的模拟。当 $b > 0$, 则程序还可以继续往下走: $\gcd(b, a \% b) = \gcd(a, b)$ 。这时假设存在一对整数 x, y , 使得其一定会满足 $b \times x + (a \% b) \times y = \gcd(b, a \% b)$, 因为 $a \% b = a - b[a/b]$, 所以有以下的推导:

$$\begin{aligned}
& b \times x + (a \% b) \times y \\
& = \gcd(b, a \% b) \\
& = bx + (a - b \lfloor a/b \rfloor)y \\
& = ay - b(x - \lfloor a/b \rfloor y)
\end{aligned}$$

这个时候令 $x' = y, y' = x - \lfloor a/b \rfloor y$ ，再结合一开始的原式子，就得出：

$$ax' + by' = \gcd(a, b)$$

因为欧几里得算法的实现是递归的，而我们已经推出其中一个递归过程的实现，那么其他的递归过程就可以借助数学归纳法，一层层地向上推，必然会得出最终结论。

证毕。

推论

1. 如果一个数 m 满足： $ax + by = m$ ，那么这个 m 一定是 $\gcd(a, b)$ 的倍数，如果不是，那么原等式无解。
2. 对于一个经典方程 $ax + by = 1$ ，利用裴蜀定理，我们有： $\gcd(a, b) = 1$ ，即 a, b 一定互质。

应用

1. 设 a, b, c 为任意整数， $g = \gcd(a, b)$ ，若方程 $ax + by = g$ 的一组整数解为 (x_0, y_0) ，则当 c 是 g 的倍数时， $ax + by = c$ 的一组解是 $(x_0 c/g, y_0 c/g)$ 。当 c 不是 g 的倍数时无整数解。
2. 设 a, b, c 为任意整数，若方程 $ax + by = c$ 的一组整数解为 (x_0, y_0) ，则它的任意整数解都可以写成 $(x_0 + kb', y_0 - ka')$ ，其中 $a' = a/\gcd(a, b)$ ， $b' = b/\gcd(a, b)$ ， k 取任意整数。

证明：假设存在两组解 (x_1, y_1) 和 (x_2, y_2) ，则有 $ax_1 + by_1 = ax_2 + by_2 = \gcd(a, b)$ ，变形得 $a(x_1 - x_2) = b(y_2 - y_1)$ 。假设 $\gcd(a, b) = g$ ，方程左右两边同时除以 g ，得 $a'(x_1 - x_2) = b'(y_2 - y_1)$ ，其中 $a' = a/g$ ， $b' = b/g$ 。注意，此时 a' 和 b' 互素，因此 $x_1 - x_2$ 一定是 b' 的整数倍。设它为 kb' ，计算得 $y_2 - y_1 = ka'$ 。注意，上面的推导过程并没有用到“ $ax + by$ 的右边是什么”，因此可以得出上面的结论。

通过 **扩展欧几里得算法**，我们可以求出方程 $ax + by = \gcd(a, b)$ 的一组整数解，进而求出全部的解。

扩展欧几里得算法

最基本的作用就是：求出 $ax + by = \gcd(a, b)$ 的一组整数解

在介绍扩展欧几里得算法之前，首先介绍它的应用：

- 1、求解不定方程（即求出 $ax + by = \gcd(a, b)$ 的一组整数解）
- 2、求解模的逆元
- 3、求解线性同余方程

为什么它能应用到这几个方面呢？回到裴蜀定理：

对于这个不定方程（又称丢番图方程），如果存在一组合法的解 (x, y) ，那么一定会有 $\gcd(a, b) | m$ ，即 m 是 $\gcd(a, b)$ 的倍数。那么现在我不仅想知道到底有没有解，而是想知道在有解的情况下，这个解到底是多少。

这就是求解不定方程的过程。这个解决的算法就叫做 **扩展欧几里得算法**。

可以发现，我们求解不定方程其实就是要求解一组合法的 (x, y) ，那么根据裴蜀定理的证明（基于欧几里得算法，采用递归的数学归纳），可以发现 x, y 的互相推导的关系。

这种采取递归来求解 x, y 的方法就叫做扩展欧几里得算法。

扩展欧几里得算法的实现：

先放板子：

```
long long exgcd(long long a, long long b, long long &x, long long &y) // 返回 a, b 的最大公约数
{
    if(b == 0)
    {
        x = 1, y = 0;
        return a;
    }
    long long d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}
```

扩展欧几里得算法的实现基于裴蜀定理的证明。实质上相当于在做欧几里得算法（普通GCD）的时候对不定方程 $ax + by = m$ 的 x, y 也做了更改。所以经过扩展欧几里得算法处理过的 x, y 就已经是一组合法的可行解了。

这里需要注意一个细节，因为扩展GCD需要对 x, y 本身进行修改，所以需要在传参数的时候加取址符，这样能保证被修改。

欧几里得算法的另一个用途就是求解 $ax + by = (a, b)$ 中的 x 和 y 。

在欧几里得算法里，递归部分的核心是这样的： $(a, b) = (b, a \bmod b) = (b, a - b \lfloor \frac{a}{b} \rfloor)$ ，我们将由此得出与 x, y 相关的递归关系。

根据裴蜀定理，我们可以找到四个整数 x, y, x', y' ，使得：

$$\begin{aligned} ax + by &= (a, b) \\ bx' + (a - b \lfloor \frac{a}{b} \rfloor)y' &= (b, a - b \lfloor \frac{a}{b} \rfloor) \end{aligned}$$

这两个等式的右侧是相等的，于是我们得到： $ax + by = bx' + (a - b \lfloor \frac{a}{b} \rfloor)y'$ ，整理可得：

$$a(x - y') + b(y - (x' - \lfloor \frac{a}{b} \rfloor y')) = 0$$

我们希望这个等式对一切 a, b 都成立，于是：

$$\begin{aligned} x &= y' \\ y &= x' - \lfloor \frac{a}{b} \rfloor y' \end{aligned}$$

这样，要求解 x 和 y ，只需要求解 x', y' ，由于 x', y' 对应的问题规模更小，所以可以进行递归的运算。最后找一下边界条件：当 $b = 0$ 时， $(a, 0)$ 对应 $x = 1, y = 0$ 。

这种算法的思想和欧几里得算法是一致的，而且它在求出最大公约数的同时，求解了一组适于裴蜀定理的系数，所以叫做扩展欧几里得算法。

应用&推论

推论1. 设 a, b, c 为任意整数。若方程 $ax + by = c$ 的一组整数解为 (x_0, y_0) ，则它的任意整数解都可以写成 $(x_0 + kb', y_0 - ka')$ ，其中 $a' = a/\gcd(a, b)$ ， $b' = b/\gcd(a, b)$ ， k 取任意整数。

证明：将 $a' = a/\gcd(a, b)$, $b' = b/\gcd(a, b)$ 带入原不定方程即可证明，略。

推论2.设 a, b, c 为任意整数， $g = \gcd(a, b)$ 。若方程 $ax + by = g$ 的一组整数解为 (x_0, y_0) ，则当 c 是 g 的倍数时 $ax + by = c$ 的一组解是 $(x_0c/g, y_0c/g)$ ；当 c 不是 g 的倍数时无整数解。

证明：原不定方程 $ax_0 + by_0 = g$ 左右同时乘以 c/g ，可得： $a \times (x_0c/g) + b \times (y_0c/g) = g \times c/g = c$ ，得证。

欧拉函数

定义

记 $\varphi(n)$ 表示小于 n 且与 n 互素的正整数的个数，我们称之为欧拉函数。

求法

设 $n = p_1^{a_1} p_2^{a_2} \dots p_k^{a_k}$ 。用容斥原理。首先从总数 n 中分别减去是 $p_1, p_2, \dots, p_k$ 的倍数的个数（对于素数 p 来说，“与 p 互素”和“不是 p 的倍数”等价），即 $n - \frac{n}{p_1} - \frac{n}{p_2} - \dots - \frac{n}{p_k}$ ，然后加上“同时是两个素因子的倍数”的个数 $\frac{n}{p_1 p_2} + \frac{n}{p_1 p_3} + \dots + \frac{n}{p_{k-1} p_k}$ ，再减去“同时是 3 个素因子的倍数”，……——写成一个“学术味比较浓”的公式就是：

$$\varphi(n) = \sum_{S \subseteq \{p_1, p_2, \dots, p_k\}} (-1)^{|S|} \frac{n}{\prod_{p_i \in S} p_i}$$

公式已得出，可计算起来很不方便。如果直接根据公式，需要计算多达 2^k 项的代数和，甚至可能比“暴力枚举（依次判断 $1 \sim n$ 中每个数是否与 n 互素）”还要慢。

下一步并不显然。上述公式可以变形成如下的形式：

$$\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \dots \left(1 - \frac{1}{p_k}\right)$$

从而只需要 $O(k)$ 的计算时间，在刚才的基础上大大提高了效率。为什么这个式子和上一个等价呢？直接考虑新公式的“展开方式”即可。展开式的每一项是从每个括号各选一个（选1或者 $-\frac{1}{p_i}$ ），全部乘起来以后再乘以 n 得到。这不正是最初的推导过程吗？详细证明如下：

证明：

设 p 是 n 的质因子， $1 \sim n$ 中 p 的倍数有 $p, 2p, 3p, \dots, (n/p) * p$ ，共 n/p 个。同理，若 q 也是 n 的质因子，则 $1 \sim n$ 中 q 的倍有 n/q 个。如果我们把这 $n/p + n/q$ 个数去掉，那么 $p * q$ 的倍数被排除了两次，需要加回来一次。因此， $1 \sim n$ 中不与 n 含有共同质因子 p 或 q 的数的个数为：

$$n - \frac{n}{p} - \frac{n}{q} + \frac{n}{pq} = n * \left(1 - \frac{1}{p} - \frac{1}{q} + \frac{1}{pq}\right) = n \left(1 - \frac{1}{p}\right) \left(1 - \frac{1}{q}\right)$$

类似地，可以在 n 的全部质因子上使用容斥原理，即可得到 $1 \sim n$ 中不与 n 含有任何共同质因子的数的个数，也就是与 n 互质的数的个数。

证毕。

根据欧拉函数的计算式，我们只需要分解质因数，即可顺便求出欧拉函数。

如果没有给出唯一分解式 $n = p_1^{a_1} p_2^{a_2} \dots p_k^{a_k}$ ，需要用试除法依次判断 $\sqrt{n}$ 内的所有素数是否是 n 的因子。这样，则需要先生成 $\sqrt{n}$ 内的素数表。但其实并不用这么麻烦：只需要每次找到一个素因子之后把它“除干净”，即可保证找到的因子都是素数（想一想，为什么）。

```
int euler_phi(int n) {
    int m = (int)sqrt(n+0.5);
    int ans = n;
    for(int i = 2; i <= m; i++)
        if(n % i == 0) {
            ans = ans / i * (i-1);    // 先除后乘，避免类型溢出
            while(n % i == 0) n /= i;
        }
    if(n > 1) ans = ans / n * (n-1); // 最后还有素因子
    return ans;
}
```

性质

1. $\forall n > 1$ ， $1 \sim n$ 中与 n 互质的数字的和为 $\varphi(n) * n/2$ 。

证明：因为 $\gcd(n, x) = \gcd(n, n - x)$ ，所以与 n 不质的数 $x, n - x$ 成对出现，平均值为 $n/2$ 。因此，与 n 互质的数的平均值也是 $n/2$ ，进而得证。

2. 若 a, b 互质, 则 $\varphi(ab) = \varphi(a)\varphi(b)$ 。

证明: 根据欧拉函数的计算式, 对 a, b 分解质因数, 可证。这个性质可以推广到一般的函数上, 得到“积性函数”的概念。

3. 当 n 是奇数时, $\varphi(2n) = \varphi(n)$ 。

证明: 当 n 是奇数时, n 与 2 互质, 由积性函数的性质可知 $\varphi(2n) = \varphi(n)\varphi(2) = \varphi(n)$ 。

4. 设有唯一分解式 $n = \prod_{i=1}^m p_i^{c_i}$, 则 $\varphi(n) = \prod_{i=1}^m \varphi(p_i^{c_i})$ 。(此性质对所有积性函数均成立)

5. 若质数 $p|n$ 且 $p^2|n$, 则 $\varphi(n) = \varphi(n/p) * p$ 。

证明: 若 $p|n$ 且 $p^2|n$, 则 $n, n/p$ 包含相同的质因子, 只是 p 的指数不同。直接把 $\varphi(n)$ 与 $\varphi(n/p)$ 按照欧拉函数的计算公式写出, 二者相除, 商为 p , 所以上式成立。

6. 若质数 $p|n$ 且 $p^2 \nmid n$, 则 $\varphi(n) = \varphi(n/p) * (p-1)$ 。

证明: 若 $p|n$ 但 $p^2 \nmid n$, 则 $n, n/p$ 互质, 由 φ 是积性函数得 $\varphi(n) = \varphi(n/p) * \varphi(p)$, 而 $\varphi(p) = p-1$, 所以上式成立。

7. 若 $n = p^k$, 其中 p 是质数, 那么 $\varphi(n) = p^k - p^{k-1}$ 。

证明: 易知, 小于 n 且能整除 n 的数字有 p^{k-1} 个, 根据欧拉函数的定义 (加上容斥原理) 可知上式显然成立。

8. 对于任意的正整数 n , 有 $\sum_{d|n} \varphi(d) = n$ 。

证明: 设 $f(n) = \sum_{d|n} \varphi(d)$ 。可得, 如果 n, m 互质, 则有

$$\begin{aligned} f(nm) &= \sum_{d|nm} \varphi(d) \\ &= \left(\sum_{d|n} \varphi(d) \right) * \left(\sum_{d|m} \varphi(d) \right) \\ &= f(n) * f(m) \end{aligned}$$

上式的一个补充解释: 利用欧拉函数的积性性质, 再对上式做因式展开即可发现上式中第一个等号后的公式和第二个等号后的公式是等价的。因为 n, m 互质, 所以 $d|nm$ 的所有因子可以分解为 $d|n$ 和 $d|m$ 两部分。

所以， $f(n) = \sum_{d|n} \varphi(d)$ 是一个积性函数。对于 n 单个质因子， $f(p^k) = \sum_{d|p^k} \varphi(d) = \varphi(1) + \varphi(p) + \varphi(p^2) + \dots + \varphi(p^k)$ 是一个等比数列求和再加 1，结果为 p^k 。所以， $f(n) = \prod_{i=1}^m f(p_i^{k_i}) = \prod_{i=1}^m p_i^{c_i} = n$ ，上式成立。

蓝桥杯真题

2023次方（蓝桥杯C/C++2023A组国赛）

欧拉降幂裸题，鉴于我网上也没有找到一个完整的证明欧拉降幂，下面我简要证明一下。

首先结论是：

$$a^b \equiv a^{b \bmod \varphi(p) + \varphi(p)} \pmod{p}$$

其中要保证 a, p 互质，当然 a, p 不互质结论也是成立的，不过下面就只证明 a, p 互质的情况，然后剩下的留给大家自己去证明。

我们知道欧拉定理如下：

$$a^{\varphi(p)} \equiv 1 \pmod{p}$$

那么我们假设现在有

$$b = k \times \varphi(p) + d, (k, d \in \mathbb{N}, d < \varphi(p))$$

接着我们去计算：

$$\begin{aligned} a^b &\equiv a^{k \times \varphi(p) + d} \equiv \prod_{i=1}^k a^{\varphi(p)} \times a^d \\ &\equiv \prod_{i=1}^k 1 \times a^d \equiv a^d \pmod{p} \end{aligned}$$

接着我们知道

$$b \equiv d \pmod{\varphi(p)}$$

那么再对欧拉定理左右两边同时乘以 a^b ，得到

$$a^b \equiv a^{\varphi(p)+b} \equiv a^{b \bmod \varphi(p)+\varphi(p)} \pmod{p}$$

证毕。

那么这个题一看就知道2, 2023是互质的，直接套用上面的结论即可。

也就是因为我们2上面的幂次太大了，所以我们相当于先用快速幂求一下上面的幂次，然后再用欧拉降幂求一下最终的答案即可，别忘了再把欧拉函数求出来。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 3000;
int prime[N], phi[N], cnt=0, p=2023;
bool vis[N];
void Euler(int n)
{
    for(int i=2; i<=n; i++)
    {
        if(!vis[i])
        {
            prime[++cnt]=i;
            phi[i]=i-1;
        }
        for(int j=1; j<=cnt; j++)
        {
            if(i*prime[j]>n) break;
            vis[i*prime[j]]=1;
            if(i%prime[j]==0)
            {
                phi[i*prime[j]]=prime[j]*phi[i];
                break;
            }
            else phi[i*prime[j]]=phi[i]*(prime[j]-1);
        }
    }
}
int qpow(int a, int b, int mod)
{
    int base=a;
```

```

int ans=1;
while(b)
{
    if(b&1)ans=ans%mod*base%mod;
    base=base%mod*base%mod;
    b>>=1;
}
return ans;
}
int main()
{
    Euler(p);
    int b=2023;
    for(int i=2022;i>=3;i--) b=qpow(i,b,phi[p]);
    cout<<qpow(2,b%phi[p]+phi[p],p);
}
//869

```

矩阵求和（蓝桥杯2018C/C++B组国赛）

简要的说题目就是要求

$$\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j)^2, (1 \leq n \leq 10^7)$$

直接 $O(n^2)$ 求解肯定超时，我们要考虑如何化简，实际上有 $\gcd(i, j)$ 这种的，都可以先去想想怎么转换成欧拉函数，我们可以对上面的式子化简如下：

$$\begin{aligned}
 & \sum_{i=1}^n \sum_{j=1}^n \gcd(i, j)^2 \\
 &= \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n [\gcd(i, j) = k] \times k^2 \\
 &= \sum_{k=1}^n k^2 \times \sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = k]
 \end{aligned}$$

接着我们可以发现进行如下化简：

$$\begin{aligned} & \sum_{k=1}^n k^2 \times \sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = k] \\ &= \sum_{k=1}^n k^2 \times \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{n}{k} \rfloor} [\gcd(i, j) = 1] \end{aligned}$$

为什么上面的化简成立，也就是说为什么

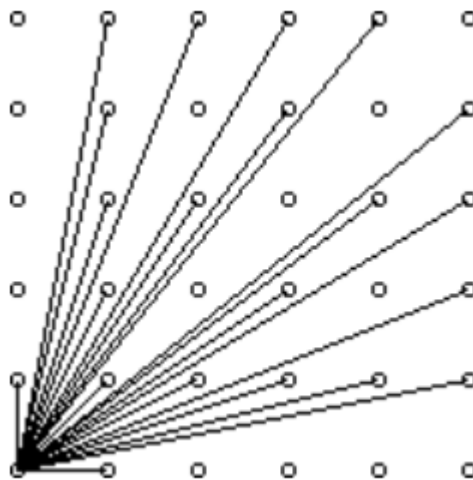
$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = k] = \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{n}{k} \rfloor} [\gcd(i, j) = 1]$$

因为我们知道如果 $\gcd(i, j) = 1$ 那么 $\gcd(k \times i, k \times j) = k$ ，所以上面右边的式子相当于是把 (i, j) 放缩了一下，通过这个 (i, j) 实际上还是依然能够求出之前的 $\gcd(i, j) = k$ 的

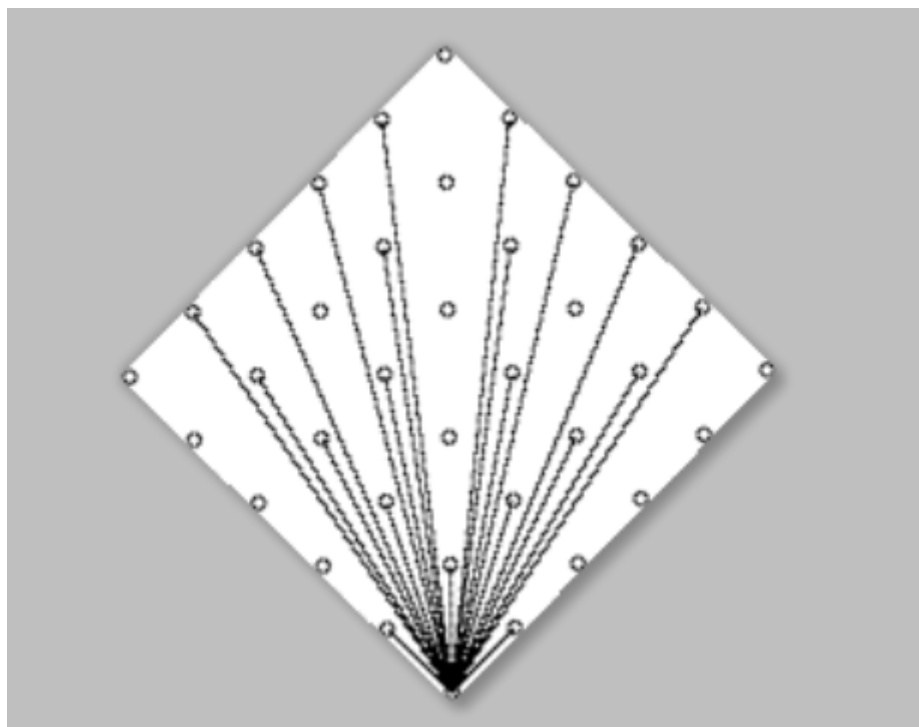
接着我们考虑欧拉函数，实际上欧拉函数可以如下表示：

$$\varphi(n) = \sum_{i=1}^n [\gcd(i, n) = 1]$$

我们再考虑把 $n \times n$ 平面内的坐标 (i, j) 满足 $\gcd(i, j) = 1$ 的点与 $(0, 0)$ 连线起来如下图所示：



可以发现对于每一条竖线他的值都是 $\varphi(i)$ ，对于每一条横线也是，实际上把图像旋转 45° ，如下图



发现这就是一个对称的图案，然后(1,1)又多算了一次所以最后的答案要减去它，所以最后我们可以得到：

$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] = 2 \times \sum_{i=1}^n \varphi(i) - 1$$

把上面这个式子再带回到原来的式子中：

$$\sum_{k=1}^n k^2 \times \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{n}{k} \rfloor} [\gcd(i, j) = 1] = \sum_{k=1}^n k^2 \times (2 \times \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \varphi(i) - 1)$$

然后我们可以线性的先把欧拉函数筛出来，做一个欧拉函数的前缀和，于是我们直接 $O(n)$ 枚举即可了。

```
#include <bits/stdc++.h>
#define ll long long

using namespace std;

const int maxn = 1e7 + 5, mod = 1e9 + 7;

int pr[maxn], ntp[maxn], phi[maxn], cnt = 0;
ll phs[maxn];
```

```

void prime(int m){
    ntp[1] = 1;
    phi[1] = 1;
    for(int i = 2; i <= m; i ++){
        if(!ntp[i]){
            pr[++ cnt] = i;
            phi[i] = i - 1;
        }
        for(int j = 1; i * pr[j] <= m && j <= cnt; j ++){
            ntp[i * pr[j]] = 1;
            if(i % pr[j] == 0){
                phi[i * pr[j]] = phi[i] * pr[j];
                break;
            }else phi[i * pr[j]] = phi[i] * phi[pr[j]];
        }
    }
}

int main(){
    int n;
    scanf("%d", &n);
    prime(n);
    for(int i = 1; i <= n; i ++) phs[i] = (phs[i - 1] + phi[i])
% mod;
    ll ans = 0;
    for(ll i = 1; i <= n; i ++) ans = (ans + i * i * (2 * phs[n
/ i] - 1) % mod) % mod;
    printf("%lld", ans);
    return 0;
}

```

习题

两个闹钟

题目要求的是 ' $n + ax = m + by$ ', 求 ' $x \geq 0$ ', ' $y \geq 0$ ' 的最小解 ' (x, y) '。

对该式进行转化: ' $ax - by = m - n$ ', 如此就可以用扩欧解决了。

需要注意的是, 这里写成 ' $ax + by = m - n$ ' 也可以的。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll a, b, n, m;
ll exgcd(ll a, ll b, ll&x, ll&y) {
    if(b==0) {
        x = 1; y = 0;
        return a;
    }
    ll d = exgcd(b, a%b, y, x);
    y -= a/b*x;
    return d;
}

int main(){
    cin >> n >> m >> a >> b;
    ll x, y, g, c = m - n;
    //b = -b;    // ax - by = m - n 这里不对 b 取负也没关系, 因为
    // y 可以取负
    g = exgcd(a, b, x, y);
    if(c % g == 0) {
        // 得到 ax+by=c 的一组初始解
        ll x0 = x*(c/g), y0 = y*(c/g);
        ll t = b/g;    // x 的整数解步长
        x0 = (x0%t + t)%t;    // x0 的最小非负整数解
        y0 = (n+a*x0-m)/b;    // 利用原式求出 y0
        //cout << t << endl;
        while(y0 < 0) {    // 找到第一个 y 的非负整数解即可
            x0 += t;    // x 累加 t, 恒大于等于 0
        }
    }
```



```

        y0 = (n+a*x0-m)/b;
    }
    cout << n+a*x0;
} else cout << "Impossible";
return 0;
}

```

直线上的点

对原等式进行转化: ' $ax + by = -c$ ', 如此, 可以套扩展欧几里德算法。

如果 ' c ' 不是 ' $\gcd(a, b)$ ' 的倍数, 则无解。

如果是, 求出一组基础解后, 按直线性质进行暴力枚举即可, 直至离开指定区域范围即可。

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll a, b, c, x_1, x_2, y_1, y_2;
ll exgcd(ll a, ll b, ll&x, ll&y) {
    if(b==0) {
        x = 1; y = 0;
        return a;
    }
    ll d = exgcd(b, a%b, y, x);
    y -= a/b*x;
    return d;
}

int main(){
    cin >> a >> b >> c >> x_1 >> x_2 >> y_1 >> y_2;
    ll x, y, g;
    g = exgcd(a, b, x, y);
    if(c % g == 0) {
        // 得到 ax+by=-c 的一组初始解
        ll x0 = x*(-c/g), y0 = y*(-c/g);
        if(x0 < x_1) { // x0 转成到 >= x_1
            ll t = ceil(1.0*(x_1-x0)/b*g);

```

```

        x0 += t*b/g; y0 -= t*a/g;
    }
    if(x0 > x_2) { // x0 转成到 <= x_2
        ll t = int((x_2-x0)/b*g);
        x0 += t*b/g; y0 -= t*a/g;
    }
    vector<pair<ll,ll>> ans;
    int k = 0, cnt; // 全部解: x0+k*b/g      y0-k*a/g
    while(1) {
        cnt = 0;
        ll xn = x0+k*b/g, yn = y0-k*a/g;

        if(xn>=x_1&&xn<=x_2&&yn>=y_1&&yn<=y_2)
            ans.push_back({xn,yn});
        else cnt++;
        xn = x0-k*b/g, yn = y0+k*a/g;
        if(k&&xn>=x_1&&xn<=x_2&&yn>=y_1&&yn<=y_2)
            ans.push_back({xn,yn});
        else cnt++;
        if(cnt == 2) break; // 已经出了区域, 此后无解
        k++;
    }
    sort(ans.begin(), ans.end());
    for(auto p : ans)
        cout << "("<<p.first<<","<<p.second<<")\n";
    if(ans.empty()) cout << "No integer point in this
zone";
    } else cout << "No integer point";
    return 0;
}

```